



SAPIENZA
UNIVERSITÀ DI ROMA

Improving Attention-Enhanced Reservoir Computing with Intrinsic Plasticity

Faculty of Ingegneria dell'informazione, informatica e statistica
Applied Computer Science and Artificial Intelligence

Emanuele Segatori

ID number 1932373

Advisor

Prof. Danilo Avola

Co-Advisor

Dr. Daniele Pannone

Academic Year 2025/2026

Thesis not yet defended

Improving Attention-Enhanced Reservoir Computing with Intrinsic Plasticity
Bachelor's thesis. Sapienza University of Rome

© 2026 Emanuele Segatori. All rights reserved

This thesis has been typeset by L^AT_EX and the Sapthesis class.

Author's email: segatori.1932373@studenti.uniroma1.it

*Dedicated to
my family and friends.*

Abstract

While the Transformer architecture has dominated the machine learning field this decade, there is a growing search for more efficient architectures during both training and inference. Reservoir computing, while remaining a strong class of models for time series prediction, struggles with discrete data like text. Attention-enhanced reservoir computing (AERC) tries to overcome this limitation using attention only on a fixed size reservoir, avoiding a growing KV cache. In this paper, we integrate Intrinsic Plasticity to the fixed reservoir to improve the AERC architecture and test it on a wider set of tasks. We find TODO

Contents

1	Introduction	1
2	Literature Review	3
2.1	Reservoir Computing	3
2.2	Intrinsic Plasticity	4
2.3	Attention-Enhanced RC	5
3	Methodology	7
3.1	AERC architecture implementation	7
3.2	IP implementation	7
3.3	IP and RMSNormalization	7
3.4	Hyperparameter Grid Search	7
4	Results	9
5	Discussion	11
6	Conclusion	13
	Bibliography	15

Chapter 1

Introduction

In recent years there has been a growing search for efficient, subquadratic alternatives to Transformer, to make training and inference cheaper. The main approaches use a lighter variant of attention, such as Native Sparse Attention [20], or linear, recurrent architectures, such as Mamba[6] or RWKV[15].

One architecture that has attracted interest for sequence modeling within the class of recurrent neural networks is the Reservoir Computing (RC) model as defined in [16], discovered independently by Jaeger[7] and Maass[13], which uses a fixed recurrent reservoir for non-linear temporal expansion of the inputs, and a linear layer to extract meaning for the high-dimensional data. The promising aspects of RC are in the fixed, recurrent, non-linear reservoir and the trainable linear output layer, which can be trained using Ridge Regression. In addition, the use of a physical reservoir makes this process even more efficient, while increasing the capabilities of the model[1].

Building upon the RC model there have been studies to expand its capabilities using multiple reservoirs [4], replacing the reservoir with a direct nonlinear feature-expansion model[5], enforcing the Echo State Property for the reservoir connection matrix[7], using leaky neurons[8], feedback connections[12] and Intrinsic Plasticity(IP)[18][16]. While these approaches have improved the base RC model, which shows state of the art results in time series applications[3], they still struggle with discrete, semantically rich data.

For this reason, an Attention-Enhanced Reservoir Computing (AERC) model has been proposed [10], using dynamically generated attention weights, greatly improving the performance of the model with discrete data while still leveraging the cheap recurrence of the fixed reservoir.

In this thesis we integrate the IP mechanism in the AERC model reservoir, showing no meaningful performance improvements, and then implement gradient based finetuning for the IP bias and gain showing downstream performance (+3% validation loss) using a marginal number of trainable parameters (+0.3%).

Chapter 2

Literature Review

2.1 Reservoir Computing

Models in the Reservoir Computing[7] family follow a simple structure, which is composed of:

1. an input layer $\mathbf{W}_{in}$ that randomly projects input $\mathbf{x}_t$ in the reservoir and is not trained
2. a reservoir described whose dynamics are described by the adjacency matrix $\mathbf{W}_r$ which is not trained
3. a trained readout $\mathbf{W}_{out}$ which uses the reservoir states r_i to predict the next state r_{i+1}

The internal dynamics of the reservoir can then be described by the function:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{in}\mathbf{x}_t + \mathbf{W}_r\mathbf{h}_{t-1})$$

The vector $\mathbf{h}_t$ is then used to express the output $\mathbf{y}_{t+1}$ using a linear transformation

$$\mathbf{y}_{t+1} = \mathbf{W}_{out}\mathbf{h}_t$$

and $\mathbf{W}_{out}$ is trained in a supervised (or self-supervised) way using least-square regression with Tikhonov regularization, commonly called ridge regression

$$\min_{\beta} \|y - x\beta\|^2 + \lambda\|\beta\|^2$$

Using an analytic solution is an extremely compelling part of RC, using computational resources and data more efficiently, and making the model more stable both during training and deployment, especially for long term behaviors like modeling a chaotic time-series. It also guarantee good generalization and simpler, faster training iterations.

2.2 Intrinsic Plasticity

The Intrinsic Plasticity (IP) mechanism, first introduced in [18], takes inspiration from a biological phenomenon called *homeostatic plasticity*, through which neurons autonomously adapt to stimuli. It is shown that neurons with an exponential firing rate distribution maximize information flow, and therefore neurons try to match such a distribution in-between their biological constraints. As a biological learning rule it differs from other commonly known phenomenon such as hebbian learning and synaptic plasticity because it modify the neuron properties rather than its connections (synapses) as shown in [2].

The objective of the IP learning rule is the maximization of entropy in the neuron output distribution, which is equivalent to the maximization of information flowing through it. Because we use the hyperbolic tangent nonlinearity, this means pushing neurons output towards a gaussian distribution

$$p_{\text{norm}}(y) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y - \mu)^2}{2\sigma^2}\right)$$

of mean μ , which is an hyperparameter.

The Gaussian is the maximum-entropy distribution under a fixed mean and variance constraint, making it a principled target for neurons whose output is bounded in $[-1, 1]$. The IP rule achieves this by locally adapting two per-neuron parameters: a gain a , which controls the slope of the activation function, and a bias b , which shifts its operating point. The adaptation is performed online by minimizing the Kullback–Leibler divergence $D_{\text{KL}}(\tilde{p} \parallel p_{\text{norm}})$ between the empirical output distribution $\tilde{p}(y)$ and the desired Gaussian target, via stochastic gradient descent.

The derived learning rules from [16] to obtain this objective are

$$\Delta b = \eta \left(1 - \left(2 + \frac{1}{\mu} \right) y + \frac{y^2}{\mu} \right)$$

$$\Delta a = \frac{\eta}{a} + \Delta b \cdot x$$

where η is the learning rate and $y = \tanh(ax + b)$ is the current neuron output. Crucially, these are purely local rules: Δb depends only on the neuron’s own output y , and Δa additionally depends on the pre-activation input x , with no backpropagation required. This makes IP biologically plausible and computationally lightweight, as each neuron adapts independently and in an unsupervised manner. The gain a effectively rescales the neuron’s sensitivity to its inputs, while the bias b shifts the distribution to match the target mean, together shaping the output statistics toward the desired Gaussian.

This means that our RC dynamics become

$$\text{pre_act} = \mathbf{W}_{in} \mathbf{x}_t + \mathbf{W}_r \mathbf{h}_{t-1}$$

$$\mathbf{h}_t = \tanh(a \cdot \text{pre_act} + b)$$

where a and b are the bias and gain added by the IP mechanism.

This mechanism is especially useful in reservoir computing, as the reservoir is usually fixed and only governed by few hyperparameters, mostly the spectral radius. Using IP provides a principled method of improving the reservoir output without compromising the analytical training. [16] demonstrated that applying IP pre-training to the reservoir consistently improved downstream performance, modifying the internal dynamics towards a more informative regime.

2.3 Attention-Enhanced RC

The Attention-Enhanced RC architecture description

Chapter 3

Methodology

3.1 AERC architecture implementation

The specific architecture chosen follow closely from the AERC original paper [10] where the author trained multiple architectures of variable size, with the biggest having 155000 trainable parameters. To compare the results we adopt the same architecture as described in the study, a vocabulary size of $V = 65$ based on the tiny_shakespeare [9] dataset, an embedding dimension $d_e = 16$ and reservoir size N of 160. The model is trained to predict the next character in a sequence by minimizing the cross-entropy loss

$$H(y, \hat{y}) = - \sum_i y_i \log(\hat{y}_i)$$

where $\hat{y}$ are the output logits and y the true label. We apply the softmax function to obtain the output probabilities (not explicitly stated in the code, Pytorch[14] applies softmax implicitly). The training is done using backpropagation and the AdamW optimizer[11], a standard variant of stochastic gradient descent. Many batches of size 32 are used for efficiency.

3.2 IP implementation

3.3 IP and RMSNormalization

3.4 Hyperparameter Grid Search

Chapter 4

Results

Text for the results.

Chapter 5

Discussion

The use of a RC class model was driven by the efficiency of the architecture both in terms of computational resources and limited dataset size, however it's also interesting the superior performance exhibited by RC models in chaotic time-series prediction, often superior to more established architectures such as Transformers[19]. This is why the RC paradigm remains interesting even under the famous Bitter Lesson[17] line of thinking, meaning that more scalable methods outperform hand-crafted alternatives.

Chapter 6

Conclusion

Text for the conclusion.

Bibliography

- [1] BRUNNER, D., SORIANO, M. C., MIRASSO, C. R., AND FISCHER, I. Parallel photonic information processing at gigabyte per second data rates using transient states. *Nature Communications*, **4** (2013). Available from: <https://api.semanticscholar.org/CorpusID:14114724>.
- [2] DESTEXHE, A. AND MARDER, E. Plasticity in single neuron and circuit computations. *Nature*, **431** (2004), 789. doi:10.1038/nature03011.
- [3] FERTÉ, T., DUTARTRE, D., HEJBLUM, B., GRIFFIER, R., JOUHET, V., THIÉBAUT, R., LEGRAND, P., AND HINAUT, X. Reservoir computing for short high-dimensional time series: An application to sars-cov-2 hospitalization forecast. In *Proceedings of the 41st International Conference on Machine Learning*, vol. 235 of *Proceedings of Machine Learning Research*, pp. 13570–13591 (2024).
- [4] GALLICCHIO, C. AND MICHELI, A. Deep reservoir computing: A critical analysis. In *The European Symposium on Artificial Neural Networks* (2016). Available from: <https://api.semanticscholar.org/CorpusID:27054014>.
- [5] GAUTHIER, D. J., BOLLT, E., GRIFFITH, A., AND BARBOSA, W. A. S. Next generation reservoir computing. *Nature Communications*, **12** (2021). Available from: <http://dx.doi.org/10.1038/s41467-021-25801-2>, doi:10.1038/s41467-021-25801-2.
- [6] GU, A. AND DAO, T. Mamba: Linear-time sequence modeling with selective state spaces (2024). Available from: <https://arxiv.org/abs/2312.00752>, arXiv:2312.00752.
- [7] JAEGER, H. The “echo state” approach to analysing and training recurrent neural networks-with an erratum note. *Bonn, Germany: German national research center for information technology gmd technical report*, **148** (2001), 13.
- [8] JAEGER, H., LUKOŠEVIČIUS, M., POPOVICI, D., AND SIEWERT, U. Optimization and applications of echo state networks with leaky- integrator neurons. *Neural Networks*, **20** (2007), 335. Echo State Networks and Liquid State Machines. Available from: <https://www.sciencedirect.com/science/article/pii/S089360800700041X>, doi:<https://doi.org/10.1016/j.neunet.2007.04.016>.

- [9] KARPATY, A. char-rnn. <https://github.com/karpathy/char-rnn> (2015). Tiny Shakespeare dataset used in the char-rnn project.
- [10] KÖSTER, F., KANNO, K., OHKUBO, J., AND UCHIDA, A. Attention-enhanced reservoir computing (2024). Available from: <https://arxiv.org/abs/2312.16503>, arXiv:2312.16503.
- [11] LOSHCHILOV, I. AND HUTTER, F. Decoupled weight decay regularization (2019). Available from: <https://arxiv.org/abs/1711.05101>, arXiv:1711.05101.
- [12] LUKOSEVICIUS, M. Echo state networks with trained feedbacks (2007).
- [13] MAASS, W., NATSCHLÄGER, T., AND MARKRAM, H. Real-time computing without stable states: A new framework for neural computation based on perturbations. *Neural Computation*, **14** (2002), 2531. doi:10.1162/089976602760407955.
- [14] PASZKE, A., ET AL. Pytorch: An imperative style, high-performance deep learning library (2019). Available from: <https://arxiv.org/abs/1912.01703>, doi:10.48550/arXiv.1912.01703.
- [15] PENG, B., ET AL. Rwkv: Reinventing rnns for the transformer era (2023). Available from: <https://arxiv.org/abs/2305.13048>, arXiv:2305.13048.
- [16] SCHRAUWEN, B., WARDERMANN, M., VERSTRAETEN, D., STEIL, J. J., AND STROOBANDT, D. Improving reservoirs using intrinsic plasticity. *Neurocomput.*, **71** (2008), 1159–1171. Available from: <https://doi.org/10.1016/j.neucom.2007.12.020>, doi:10.1016/j.neucom.2007.12.020.
- [17] SUTTON, R. S. The bitter lesson. <http://www.incompleteideas.net/IncIdeas/BitterLesson.html> (2019). Accessed: 2026-06-27.
- [18] TRIESCH, J. A gradient rule for the plasticity of a neuron’s intrinsic excitability. In *Artificial Neural Networks: Biological Inspirations – ICANN 2005* (edited by W. Duch, J. Kacprzyk, E. Oja, and S. Zadrozny), pp. 65–70. Springer Berlin Heidelberg, Berlin, Heidelberg (2005). ISBN 978-3-540-28754-4.
- [19] VASWANI, A., SHAZEER, N., PARMAR, N., USZKOREIT, J., JONES, L., GOMEZ, A. N., KAISER, L., AND POLOSUKHIN, I. Attention is all you need (2023). Available from: <https://arxiv.org/abs/1706.03762>, arXiv:1706.03762.
- [20] YUAN, J., ET AL. Native sparse attention: Hardware-aligned and natively trainable sparse attention. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (edited by W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar), pp. 23078–23097. Association for Computational Linguistics, Vienna, Austria (2025). ISBN 979-8-89176-251-0. Available from: <https://aclanthology.org/2025.acl-long.1126/>, doi:10.18653/v1/2025.acl-long.1126.